

TxFE/MxFE Eval App Extras - User Guide

Contents

- Revision History2
- Overview.....3
- Installation.....3
- Building and Executing on MicroZed4
 - Building the Standalone App4
 - Building the Xtras App5
- CLI Commands6
 - Command Format.....6
 - Command Types6
 - TX Commands.....6
 - RX Commands7
- MicroZed Use Case Example.....7
- Working with SSH and SCP9
- Loading an FPGA Image10
- MATLAB Usage Examples11
 - MATLAB Sample 1 – Create tone with xtras app.....11
 - MATLAB Sample 2 – Create tone with MATLAB.....13
- Bash Shell Scripts.....17
 - Standalone App Shell Script17
 - Xtras Tx Command Shell Script.....17
 - Xtras Rx Command Shell Script18
- Troubleshooting23

Evaluation Warning: The document was created with Spire.Doc for Python.

Revision History

Rev	Date	Description
1.0	6/26/2020	Initial version.
1.1	8/04/2020	Updates for API 1.0.5 Add MATLAB section
1.2	11/10/2020	Updates for API 1.0.6
1.3	12/03/2020	Fixed some version 1.0.5 references
1.4	12/21/2020	Updates for API 1.0.7
1.5	03/05/2021	Updates for API 1.1.0

Evaluation Warning: The document was created with Spire.Doc for Python.

Overview

This document describes the MxFE Eval App Extras console application that provides simple tone generation and capture functionality for an ADS9 based evaluation platform. It runs directly on the MicroZed under Linux and has a command line interface. The intent of this app is to provide simple methods for working with the MxFE after it has been configured. For Tx, tone vectors can be created or read in from files and loaded into the ADS9 platform controller for playback. Similarly, for Rx, capture data is collected and written out to files which can be analyzed with an external program such as MATLAB. All data is in terms of DAC and ADC code samples.

An example of a tx and rx command is shown below. This is how the commands would be executed directly on the MicroZed.

```
ad9082_xtra tx --stone 368.64 10 8192 -3.0
```

```
ad9082_xtra rx --cap capture 8192
```

A typical usage scenario is to first configure the MxFE and ADS9 FPGA platform, using the standalone application, then use the Eval App Extras (xtras) program to generate tones and capture data. The xtras app can be executed over and over once the MxFE is configured. It can be executed directly on the MicroZed using a terminal emulator, such as Putty, or programmatically from the PC using ssh/scp commands. One could write MATLAB or Python scripts to load vectors and acquire data which can then be processed and analyzed.

MxFE variants AD9177 and AD9207 are DAC and ADC variants only, respectively. Therefore, use tone generation or capture data accordingly.

The xtras application is distributed as C source code and is part of the API package. It is intended to be used as a baseline for user customization. This guide provides installation and build instructions, command line arguments and an example use case.

Evaluation Warning: The document was created with Spire.Doc for Python.

Installation

The xtras application is installed as part of the API package. The MxFE API source code package is available for download via www.analog.com. Please see the [Software Request Form](#) for obtaining an installer.

There are 3 primary folders in the package. They contain the API, the standalone app and the xtras app. All are independent. For example, the standalone and xtras app each have a copy of the API source code in their folder structure.

Source code packages from API installer:

- | | |
|----------------|--|
| 1. ad9082_api | API source code only |
| 2. ad9082_app | Standalone Linux application for device configuration on ADS9 platform |
| 3. ad9082_xtra | Linux application for loading vectors and capturing data on ADS9 platform. |

The installer places the source code in the following folder by default:

```
C:\Analog Devices\AD9082_API-Rel1.1.0\src\ad9082_api  
C:\Analog Devices\AD9082_API-Rel1.1.0\src\ad9082_app  
C:\Analog Devices\AD9082_API-Rel1.1.0\src\ad9082_xtra
```

Building and Executing on MicroZed

This section describes how to build the standalone and xtras applications on the MicroZed. It is very straight forward. Here are the basic steps:

1. Verify a connection with the MicroZed
2. Copy the source code from the PC to the MicroZed

3. Run 'make' in the *app_ads9* sub-folder
4. Execute the program

Before starting, make sure you can access and log into the MicroZed. You can use a terminal emulator, such as Putty, or ssh if a Bash shell is available on your PC. This document will use ssh/scp for executing commands and copying files to/from the MicroZed.

Note:

Establishing an access key for the MicroZed eliminates the requirement to sign in for every ssh/scp command. This is very useful for initiating commands from the PC. See the section Working with SSH and SCP for more information.

The login credentials for the MicroZed are:

IP address of MZ: 192.168.0.10
User: root
PW: analog

Building the Standalone App

The following commands (from a PC Bash shell) copy the standalone application to the MicroZed and builds it.

```
scp -r /c/Analog\ Devices/AD9082_API-Rel1.1.0/src/ad9082_app root@192.168.0.10:ad9082_app_110  
ssh root@192.168.0.10 'cd ~/ad9082_app_110/app_ads9; make clean all'
```

The folder on the MicroZed has "_110" appended to distinguish it from other versions. This is not required.

Here's an example of running the standalone for use case #3. Make sure an ADS9 FPGA image has been loaded or *chip-to-chip* errors will occur. See the section on Loading an FPGA Image below.

Evaluation Warning: The document was created with Spire.Doc for Python.

```
ssh root@192.168.0.10 '~/ad9082_app_110/app_ads9/debug/ad9082_app 3 1 1'
```

If you are not using Bash, then WinSCP can be used to transfer the files from the PC to the MicroZed. Then, just Putty into the MicroZed, cd to the app_ads9 folder and run the make there. Perform similar operations for executing the program.

Building the Xtras App

The following commands (from a PC Bash shell) copies the xtras application to the MicroZed and builds it.

```
scp -r /c/Analog\ Devices/AD9082_API-Rel1.1.0/src/ad9082_xtra root@192.168.0.10:ad9082_xtra  
ssh root@192.168.0.10 'cd ~/ad9082_xtra/app_ads9; make clean all'
```

Here's an example of running the xtras app to generate and load a single tone. As mentioned previously, make sure an ADS9 FPGA image has been loaded or *chip-to-chip* errors will occur. See the section on Loading an FPGA Image below. Also, make sure the device has been configured using the standalone app. The instructions for running the standalone app are shown above.

This command will generate a 10MHz complex single tone with a data rate of 368.64MHz. It will contain 8192 samples per virtual converter with -3.0dB of backoff. The default number of bits, 16, will be applied.

```
ssh root@192.168.0.10 "~/ad9082_xtra/app_ads9/debug/ad9082_xtra tx --stone 368.64 10 8192 -3.0"
```

Sample output from tx command

```
$ ssh root@192.168.0.10 "~/ad9082_xtra/app_ads9/debug/ad9082_xtra tx --stone 368.64 10 8192 -3.0"
Chip-to-chip bridge: Ready detected.
UIO devices found: uio_spi_controller, uio_spi_gpio, uio_axi_fpga, uio_i2c_controller.
UIO mapping address: 0xb6f8f000, 0xb6f8e000, 0xb6f5f000, 0xb6f8d000.
Checking JESD link status:
AD9082 JRX link0 is up. AD9082 JRX Status = 0x000f, FPGA JTX Status = 0x00
AD9082 JTX link0 is up. AD9082 JTX Status = 0x007d, FPGA JRX Status = (0x00, 0x00), EMB_ERR_CNT = 0, CRC_ERR_CNT = 0, RX_ERR_CNT = 60952
AD9082 JTX link1 is up. AD9082 JTX Status = 0x007d, FPGA JRX Status = (0x00, 0x00), EMB_ERR_CNT = 0, CRC_ERR_CNT = 0, RX_ERR_CNT = 60952

Vector Generation
000.110: ads9_hw_close(), spi_rd = 28, spi_wr = 3, total = 31
```

This command will capture 8192 samples and store the results in a series of files named captureLxCy.txt. The 'x' and 'y' values represent the link number and virtual converter, respectively, per the JESD204 link configuration.

```
ssh root@192.168.0.10 "~/ad9082_xtra/app_ads9/debug/ad9082_xtra rx --cap capture 8192"
```

Sample output from rx command. Includes RMS values from virtual converters (I/Q data)

```
$ ssh root@192.168.0.10 "~/ad9082_xtra/app_ads9/debug/ad9082_xtra rx --cap capture 8192"
Chip-to-chip bridge: Ready detected.
UIO devices found: uio_spi_controller, uio_spi_gpio, uio_axi_fpga, uio_i2c_controller.
UIO mapping address: 0xb6f4e000, 0xb6f4d000, 0xb6f1e000, 0xb6f4c000.
Checking JESD link status:
AD9082 JRX link0 is up. AD9082 JRX Status = 0x000f, FPGA JTX Status = 0x00
AD9082 JTX link0 is up. AD9082 JTX Status = 0x007d, FPGA JRX Status = (0x00, 0x00), EMB_ERR_CNT = 0, CRC_ERR_CNT = 0, RX_ERR_CNT = 14408
AD9082 JTX link1 is up. AD9082 JTX Status = 0x007d, FPGA JRX Status = (0x00, 0x00), EMB_ERR_CNT = 0, CRC_ERR_CNT = 0, RX_ERR_CNT = 14408

Read Capture Data and Format
cap_complete: 1
Capture file location: /root
Link 0:
  635.7    635.8
Link 1:
  4.9      5.0
000.649: ads9_hw_close(), spi_rd = 28, spi_wr = 3, total = 31
```

CLI Commands

Evaluation Warning: The document was created with Spire.Doc for Python. The xtras console application has a simple command line interface (CLI) for executing various commands. Currently it only supports basic Tx vector and Rx capture features. The CLI will expand once new features become available.

There's a version of xtras for each variant of the TxFe/MxFE, just like the API. For example, for AD9082 TxFe, the xtras executable name is "ad9082_xtra"

Command Format

```
ad9082_xtra <cmd type> <--cmd> <options>
```

cmd type	Device function such as TX or RX
cmd	Command
options	Command specific options

```
ad9082_xtra -h|--help
```

```
-h|--help    Display usage and command options
```

Command Types

- TX
 - stone single tone vector generation and load
 - file load vectors from file
- RX
 - cap capture samples to file. (file per virtual conv)
 - rcap raw capture to file (samples directly from FPGA platform memory)

TX Commands

Commands related to TX function of device.

```
txfe_xtra tx --stone <fd> <ft> [vecLen][backoff][nbits]
txfe_xtra tx --file <file0> [file1] [file2] ...
```

--stone Generate single tone and load vector with specified parameters

fd Data rate

ft Tone frequency

vecLen Vector length (per virtual converter) (optional, default 8192, max 1048576)

backoff Vector backoff in db (optional, default 0.0, min -100.0)

nbits Number of bits in sample (optional, default 16, range 8 to 16)

--file Load vectors from file

file0 Vector files for virtual converter 0. If only one specified, then all VCs use it. If two are specified, then VC0=file0 and VC1=file1.

file* Vector file

When the number of VCs is > number of files, then the assignment wraps. For example, assume jesd m is 4 and two files are specified:

VC0 = file0

VC1 = file1

VC2 = file0

VC3 = file1

If more files are specified than the number of VCs the extra ones are ignored.

Evaluation Warning: The document was created with Spire.Doc for Python.

RX Commands

Commands related to RX function of device.

```
txfe_xtra rx --cap <file> [nsamples]
```

```
txfe_xtra rx --rcap <file> [nsamples]
```

--cap Capture data and split into individual files

file File name prefix for split files. Append "_LxCy" where x is link (0|1) and y is virtual converter

nsamples Number of samples to capture per virtual converter (optional, default 8192, max 1048576)

--rcap Capture data and store raw samples in single file

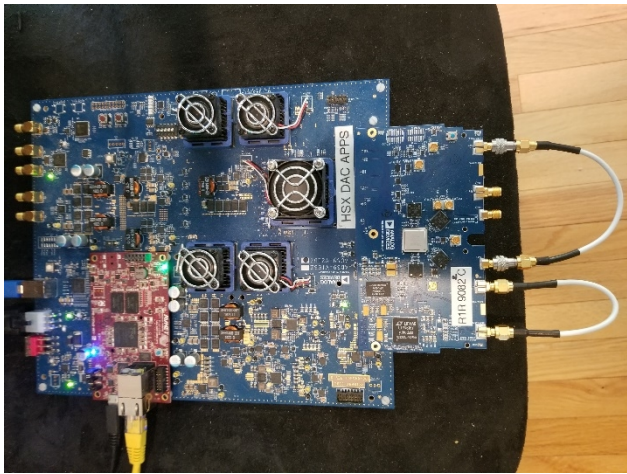
file File name to store raw samples.

nsamples Number of samples to capture per virtual converter (optional, default 8192, max 1048576)

MicroZed Use Case Example

A typical use case for using the xtras application involves configuring the part, loading vectors for Tx and capturing data from Rx. Configuring the part can be accomplished by running the standalone app. The xtras application requires ADI's development platform which consists of an ADS9 FPGA controller board.

This example uses the AD9082 evaluation board. DAC0 is looped back to ADC0 while DAC1 is looped back to ADC1. The loopback is done via the sma connectors located on the edge of the eval board. A 10MHz tone is output from the DAC (NCO shifted up) and samples are captured with the ADC (NCO shifted down). The I/Q capture data is saved off to individual files, each representing a JESD virtual converter. For this example, 4 capture files are created (2 virtual converters * 2 links)



The AD9082 is configured using the standalone application, use case #3. This sets up the device with the following configuration:

	Fs	Total Int/Dec	Fdata	Main Shift (Conv0/Conv1)	Chan Shift	# links	VConv/link (m)
Tx	5898.24	16	368.64	1842.5/1842.5	0/0	1	4
Rx	2949.12	8	368.64	1842.5/350.0	0/0	2	2

The procedure for running this xtras example is described below. Make sure both the standalone and xtras app are available on the MicroZed. See the section Building and Executing on MicroZed above for details

Evaluation Warning: The document was created with Spire.Doc for Python.

1. Configure the FPGA image on the ADS9. This can be done via DPGLite or ACE. See the section Loading an FPGA image below.

2. Remote login into the MicroZed using ssh or Putty.
user: root, pw: analog

3. Run the standalone app

```
~/ad9082_app_110/app_ads9/debug/ad9082_app 3 1 1
```

4. Create a 10MHz tone with 0.0dB backoff. Fdata is 368.64

```
~/ad9082_xtra/app_ads9/debug/ad9082_xtra tx --stone 368.64 10.0 8192 0.0
```

5. Capture 8192 samples from each ADC and create files

```
~/ad9082_xtra/app_ads9/debug/ad9082_xtra rx --cap capture 8192
```

The RMS sample values of each virtual converter are displayed in the program output. As an experiment, try setting the tone backoff to -6.0 with the ad9082_xtra_tx command and see the RMS value cut in half.

Here is some console output from running the commands:

```

root@analog:~# ~/ad9082_app_105/app_ads9/debug/ad9082_app 3 1 1
Running uc3 using hmc7044.
Chip-to-chip bridge: Ready detected.
UIO devices found: uio_spi_controller, uio_spi_gpio, uio_axi_fpga, uio_i2c_controller.
UIO mapping address: 0xb6fec000, 0xb6feb000, 0xb6fbc000, 0xb6fea000.
Found connected device board: AD9082-FMCA-EBZ-W2 Rev:C
AD9082 API v1.0.5
FPGA Image v19061100 sv19050200
Checking JESD link status:
  AD9082 JRX link0 is up. AD9082 JRX Status = 0x000f, FPGA JTX Status = 0x00
  AD9082 JTX link0 is up. AD9082 JTX Status = 0x007d, FPGA JRX Status = (0x00, 0x00), EMB_ERR_CNT = 0, CRC_ERR_CNT = 0, RX_ERR_CNT = 15660
  AD9082 JTX link1 is up. AD9082 JTX Status = 0x007d, FPGA JRX Status = (0x00, 0x00), EMB_ERR_CNT = 0, CRC_ERR_CNT = 0, RX_ERR_CNT = 15660
001.168: ads9_hw_close(), spi_rd = 778, spi_wr = 1594, total = 2372

root@analog:~# ~/ad9082_xtra/app_ads9/debug/ad9082_xtra tx --stone 368.64 10.0 8192 0.0
Chip-to-chip bridge: Ready detected.
UIO devices found: uio_spi_controller, uio_spi_gpio, uio_axi_fpga, uio_i2c_controller.
UIO mapping address: 0xb6f17000, 0xb6f16000, 0xb6ee7000, 0xb6f15000.
Checking JESD link status:
  AD9082 JRX link0 is up. AD9082 JRX Status = 0x000f, FPGA JTX Status = 0x00
  AD9082 JTX link0 is up. AD9082 JTX Status = 0x007d, FPGA JRX Status = (0x00, 0x00), EMB_ERR_CNT = 0, CRC_ERR_CNT = 0, RX_ERR_CNT = 33612
  AD9082 JTX link1 is up. AD9082 JTX Status = 0x007d, FPGA JRX Status = (0x00, 0x00), EMB_ERR_CNT = 0, CRC_ERR_CNT = 0, RX_ERR_CNT = 33612

Vector Generation
000.110: ads9_hw_close(), spi_rd = 28, spi_wr = 3, total = 31

root@analog:~# ~/ad9082_xtra/app_ads9/debug/ad9082_xtra rx --cap capture 8192
Chip-to-chip bridge: Ready detected.
UIO devices found: uio_spi_controller, uio_spi_gpio, uio_axi_fpga, uio_i2c_controller.
UIO mapping address: 0xb6ff3000, 0xb6ff2000, 0xb6fc3000, 0xb6ff1000.
Checking JESD link status:
  AD9082 JRX link0 is up. AD9082 JRX Status = 0x000f, FPGA JTX Status = 0x00
  AD9082 JTX link0 is up. AD9082 JTX Status = 0x007d, FPGA JRX Status = (0x00, 0x00), EMB_ERR_CNT = 0, CRC_ERR_CNT = 0, RX_ERR_CNT = 13752
  AD9082 JTX link1 is up. AD9082 JTX Status = 0x007d, FPGA JRX Status = (0x00, 0x00), EMB_ERR_CNT = 0, CRC_ERR_CNT = 0, RX_ERR_CNT = 13752

Read Capture Data and Format
cap_complete: 1
Capture file location: /root
Link 0:
  930.4    930.7
Link 1:
  4.9      5.0
000.650: ads9_hw_close(), spi_rd = 28, spi_wr = 3, total = 31

root@analog:~#

```

```

root@analog:~# ls capture*.txt
capture_L0C0.txt  capture_L0C1.txt  capture_L1C0.txt  capture_L1C1.txt
root@analog:~#

```

Notes:

Evaluation Warning: The document was created with Spire.Doc for Python.

- The operations are executed directly on the MicroZed from the root dir (cd ~)
- The RMS I/Q values for ADC1 are approximately 0 because of Main Shift Val (see table above)
- Capture file names are: capture_LxCy.txt where x is link (0|1) and y is vc (0|1)

Working with SSH and SCP

SSH and SCP are Secure Shell and Secure Copy protocols for executing shell commands and copying files, respectively, between a local (PC) and remote host (MZ). They are convenient to use when working with the xtras application because commands and file transfer are initiated from the PC. Scripts can be written, in Bash, Python, MATLAB, etc which run on the PC.

Setting up auto ssh login between the PC and MicroZed provides the best experience This is easily accomplished by setting up authentication keys. You can search the internet for methods of accomplishing this or try the steps outlined below.

Log into MZ and delete the ~/.ssh dir. This will clear out any previous keys:

```
# rm -rf ~/.ssh
```

Then, run the following commands from a Bash shell on PC. If first time, then run keygen. Otherwise, just copy the existing key:

```
$ ssh-keygen (first time only, select all the defaults)
$ ssh-copy-id -i ~/.ssh/id_rsa.pub root@192.168.0.10
```

Test remote login to MicroZed

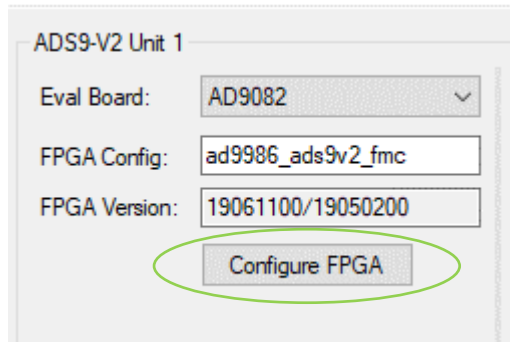
```
$ ssh root@192.168.0.10
```

There're a couple of ways to obtain a Bash shell for windows.

- Git installation has a version of Bash shell with ssh/scp utilities.
 - <https://git-scm.com/>
- MinGW contains a Bash shell and many other utilities for GNU based development on Windows. This may be a bit heavy for simple Bash requirements.
 - <https://osdn.net/projects/mingw/>

Loading an FPGA Image

For the standalone or xtras app to execute properly an FPGA image must be configured. This can be done with DPGDownloaderLite or ACE. The UI control below is part of DPGLite.



If you attempt to run the standalone or the xtras app without a configured FPGA, you will receive the following error messages. For example:

Evaluation Warning: The document was created with Spire.Doc for Python.

```
$ ssh -root@192.168.0.10 -v /ad9082_app.104/app_ads9/debug/ad9082_app.3 -1'
Running uc3 on CE board using hmc7044.
Chip-to-chip bridge: Not ready, 1, 0.
Chip-to-chip bridge: Not ready, 1, 0.
```

A future version of the xtras app will provide this capability.

MATLAB Usage Examples

This section shows examples of using the xtras library from MATLAB. Basically, the MATLAB script uses the “system” function to remote execute programs on the MicroZed and copy files to and from the PC. MATLAB can then be used to process the data.

To run the MATLAB Code, an FPGA image must be configured. This can be done with DPG DownloaderLite or ACE. For more detail on this step, see the Loading an FPGA Image section.

MATLAB Sample 1 – Create tone with xtras app

This MATLAB script will configure the AD9082 device using the standalone app for use case #3 as discussed above. It then creates a 10MHz complex single tone for DAC0/1 and configures the ADS9. Finally, it captures data and plots the time and FFT results of the looped back analog signals.

To run the MATLAB code for a single tone input, refer to the script “matlab_plotdat.m”. Within the script, the system calls can be edited to fit the preferred input.

matlab_plotdat.m

%@brief TxFE Xtras App Matlab Plot